

Lecture 5: Link Layer - Error Control Coding

EC 441 - Introduction to Computer Networking

Boston University, Spring 2026

Learning Objectives

By the end of this lecture, you should be able to:

1. Explain why error control is needed at the link layer
2. Define block codes using (n, k) notation and compute the code rate
3. Distinguish between error detection, error correction, and combined approaches
4. Compute the Hamming distance between codewords and determine $d_{\min}$ for a code
5. Apply the $d_{\min}$ theorem to determine valid (e_c, e_d) pairs
6. Analyze design tradeoffs between error detection and error correction

1 Introduction to Error Control

1.1 Why Error Control?

At the physical layer, transmitted bits may be corrupted by noise, interference, or fading. The **link layer** adds redundancy to detect and/or correct these errors before data is passed up to higher layers.

Goal: Decrease the residual bit error rate (BER) without reducing throughput too much.

Key idea: Add redundant bits to allow detection (and possibly correction) of errors.

Fundamental tradeoff: Increasing redundant bits improves error resilience but decreases throughput.

Where error control fits in the protocol stack:

Application
Transport
Network
Link Layer: Error control coding ←
Physical Layer: Raw bit transmission (with errors)

Context: In Lecture 4 we studied the wireless physical layer, where bit errors are common due to path loss, fading, and interference. Error control coding at the link layer is the first line of defense against these errors.

2 Block Codes: (n, k) Notation

2.1 Error-Control Coding Framework

Basic setup:

- Take k **data bits** (call it Data), transmit an n -bit long **codeword** (call it Word)
- There are 2^k valid codewords out of 2^n possible n -bit words
- The remaining $2^n - 2^k$ words are **non-codewords** (invalid)
- This is called a **block code**, designated as an (n, k) block code

Block Code Sizes

For a $(7, 4)$ block code:

- $k = 4$ data bits, $n = 7$ codeword bits
- $2^4 = 16$ valid codewords out of $2^7 = 128$ possible 7-bit words
- $128 - 16 = 112$ non-codewords (invalid words)
- $n - k = 3$ redundant (check) bits per codeword

2.2 Encoding and Decoding

Encoding: Calculate $\text{Word} = g(\text{Data})$ and transmit Word.

Decoding: At the receiver, calculate $\text{Data} = g^{-1}(\text{Received Word})$.

Common structure: Often, the codeword consists of the original data followed by check bits:

$$\text{Word} = \text{Data} \mid f(\text{Data}) \quad (1)$$

where $f(\text{Data})$ computes the $n - k$ redundant (check) bits and “ $\mid$ ” denotes concatenation.

This is called a **systematic code** because the original data bits appear explicitly in the codeword.

2.3 Code Rate

The **rate** of the code is:

$$R_c = \frac{k}{n} \quad (2)$$

- $R_c = 1$: No redundancy (no error protection)
- $R_c < 1$: Redundancy added (throughput reduced by factor R_c)
- Lower R_c means more redundancy and potentially better error protection

Code Rate Examples

- $(7, 4)$ code: $R_c = 4/7 \approx 0.57$ — 57% of transmitted bits carry data
- $(8, 7)$ parity code: $R_c = 7/8 = 0.875$ — very efficient, minimal protection
- $(3, 1)$ repetition code: $R_c = 1/3 \approx 0.33$ — high overhead, strong protection

3 Error Detection vs. Correction

3.1 Three Error Control Options

Given a received word, the receiver has three fundamental strategies:

Option 1 — Error correction (Forward Error Correction, FEC):

- Any received word is mapped to the *nearest* valid codeword
- No retransmission needed
- Higher overhead (more redundant bits required)

Option 2 — Error detection (Automatic Repeat reQuest, ARQ):

- Only the 2^k valid codewords are accepted
- Invalid words trigger a retransmission request
- Lower overhead but requires a feedback channel

Option 3 — Combined detection and correction:

- Some non-codewords are **corrected** (those close to a valid codeword)
- Some non-codewords are **rejected** (those not close to any valid codeword)
- All valid codewords are accepted
- Provides the best of both worlds

4 Parity Bit and Repetition Code Examples

4.1 The Parity Bit Code (Even Parity)

Construction:

- $n - k = 1$ (one check bit added)
- Send the k data bits followed by a parity bit that forces an **even number of ones** in the $(k + 1)$ -bit codeword

(4, 3) Even Parity Code

Data (3 bits)	Parity Bit	Codeword (4 bits)
000	0	0000
001	1	0011
010	1	0101
011	0	0110
100	1	1001
101	0	1010
110	0	1100
111	1	1111

Properties of the parity bit code:

- Rate: $R_c = k/(k+1)$ (very efficient, approaches 1 as k grows)
- Can detect any single-bit error (or any odd number of errors)
- Cannot correct any errors
- Cannot detect an even number of errors
- $d_{\min} = 2$ (every pair of codewords differs in at least 2 positions)

4.2 The Repetition Code

Construction:

- $k = 1$ data bit, repeated n times
- Two codewords: n zeros or n ones
- Rate: $R_c = 1/n$ (very low, high overhead)

(3,1) Repetition Code

Data	Codeword
0	000
1	111

Properties:

- Rate: $R_c = 1/3$
- $d_{\min} = 3$ (the two codewords differ in all 3 positions)
- Can detect up to 2 errors, OR correct 1 error (but not both simultaneously at maximum)

General repetition code:

- For a $(n, 1)$ repetition code: $d_{\min} = n$
- Maximum detection: $e_d = n - 1$ errors
- Maximum correction: $e_c = \lfloor (n - 1)/2 \rfloor$ errors
- Simple but extremely inefficient ($R_c = 1/n$)

5 Hamming Distance and Minimum Distance

5.1 Hamming Distance

The **Hamming distance** between two n -bit words is the number of bit positions in which they differ.

$$\text{dist}(X, Y) = \sum_{i=1}^n (x_i \oplus y_i) \quad (3)$$

where $\oplus$ denotes the XOR operation.

Computing Hamming Distance

$\text{dist}(0011, 1010) = 2$ (differ in positions 1 and 4)
 $\text{dist}(0000, 1111) = 4$ (differ in all positions)
 $\text{dist}(1010, 1010) = 0$ (identical words)
 $\text{dist}(1100, 0011) = 4$ (differ in all positions)

Properties of Hamming distance:

- $\text{dist}(X, Y) \geq 0$ (non-negative)
- $\text{dist}(X, Y) = 0$ if and only if $X = Y$
- $\text{dist}(X, Y) = \text{dist}(Y, X)$ (symmetric)
- $\text{dist}(X, Z) \leq \text{dist}(X, Y) + \text{dist}(Y, Z)$ (triangle inequality)

5.2 Minimum Distance of a Code

A **code** $\mathcal{C}$ is a set of valid n -bit codewords.

The **minimum distance** $d_{\min}$ of a code is the smallest Hamming distance between any pair of distinct codewords:

$$d_{\min} = \min_{\substack{C_i, C_j \in \mathcal{C} \\ C_i \neq C_j}} \text{dist}(C_i, C_j) \quad (4)$$

Why $d_{\min}$ matters: It completely determines the error detection and correction capability of the code. A larger $d_{\min}$ means greater separation between codewords, which means more errors can be tolerated.

Computing $d_{\min}$

Consider the code $\mathcal{C} = \{000, 011, 101, 110\}$. We compute all pairwise distances:

Pair	Distance
$\text{dist}(000, 011)$	2
$\text{dist}(000, 101)$	2
$\text{dist}(000, 110)$	2
$\text{dist}(011, 101)$	2
$\text{dist}(011, 110)$	2
$\text{dist}(101, 110)$	2

Therefore $d_{\min} = 2$. This code can detect 1 error but cannot correct any.

5.3 Visualizing Hamming Distance

Think of codewords as points in an n -dimensional binary space. The Hamming distance is the number of coordinates that differ — like a “city block” distance in binary space. The minimum distance $d_{\min}$ determines how far apart the closest pair of codewords are, which in turn determines how many errors the code can handle.

Geometric intuition:

- Each error moves the received word 1 step away from the transmitted codeword
- If codewords are far apart ($d_{\min}$ is large), more errors are needed before a received word looks like a different codeword
- The “spheres” of correctable errors around each codeword must not overlap

6 Code Power: e_c and e_d Parameters

6.1 Defining Code Power

For a given code and receiver algorithm, define:

- e_c = the number of errors **guaranteed to be corrected** successfully
- e_d = the number of errors **guaranteed to be detected** successfully (where $e_d \geq e_c$)

The pair (e_c, e_d) completely describes the error control strategy.

6.2 Receiver Algorithm

Let R be the received word and e be the apparent number of errors:

$$e = \min_{C_i \in \mathcal{C}} \text{dist}(R, C_i) \quad (5)$$

The receiver operates as follows:

- If $e = 0$: R is a valid codeword — **accept** and decode
- If $1 \leq e \leq e_c$: **correct** — map R to the nearest codeword C_i
- If $e_c < e$: **detect error** — request retransmission, discard, or flag as uncorrectable

6.3 Hamming Sphere Interpretation

Each codeword C_i has a “sphere” of radius e_c around it — all received words within Hamming distance e_c of C_i are corrected to C_i .

Key requirements:

- Correction spheres around different codewords **must not overlap** (otherwise correction is ambiguous)
- Words outside all correction spheres but within distance e_d of the nearest codeword are detected as errors
- Words more than e_d away from any codeword may go undetected (undetectable errors)

Hamming Spheres for (3,1) Repetition Code

With codewords $\{000, 111\}$ and $(e_c = 1, e_d = 1)$:

Sphere around 000: $\{000, 001, 010, 100\}$ — all corrected to 000

Sphere around 111: $\{111, 110, 101, 011\}$ — all corrected to 111

The spheres partition all of $\{0, 1\}^3$ — every word is correctable. No word falls outside both spheres, so every error pattern of 1 or fewer errors is corrected.

7 The $d_{\min}$ Theorem

7.1 Theorem Statement

The $d_{\min}$ Theorem

For a code with minimum distance $d_{\min}$, the error-correcting parameter e_c and error-detecting parameter e_d must satisfy:

$$e_c + e_d \leq d_{\min} - 1 \quad (6)$$

$$e_c \leq e_d \quad (7)$$

7.2 Proof Sketch

Why $e_c \leq e_d$?

If we can correct an error pattern, we must necessarily detect its presence first. Any correctable error is also a detectable error. Hence $e_c \leq e_d$.

Why $e_c + e_d \leq d_{\min} - 1$?

Consider two codewords C_i and C_j that achieve the minimum distance: $\text{dist}(C_i, C_j) = d_{\min}$.

Suppose for contradiction that $e_c + e_d = d_{\min}$. Then there exists a received word R on the “path” between C_i and C_j such that:

$$\begin{aligned} \text{dist}(R, C_i) &= e_c \\ \text{dist}(R, C_j) &= d_{\min} - e_c = e_d \end{aligned}$$

Now consider two scenarios:

- **If C_i was sent:** R has e_c errors. Since $e_c \leq e_c$, the receiver should *correct* R to C_i . This works correctly.
- **If C_j was sent:** R has e_d errors. Since $e_d \leq e_d$, the receiver should *detect* this error and reject R . But the receiver sees the same R and has already decided to correct it to C_i .

The receiver cannot distinguish these two cases — it sees the same received word R in both scenarios. Therefore, it cannot simultaneously guarantee correction up to e_c *and* detection up to e_d when $e_c + e_d = d_{\min}$. Hence:

$$e_c + e_d \leq d_{\min} - 1$$

7.3 Consequences of the $d_{\min}$ Theorem

(1) **Maximum error detection (set $e_c = 0$):**

$$e_d = d_{\min} - 1 \quad (8)$$

This gives the best possible pure detection capability: $(e_c, e_d) = (0, d_{\min} - 1)$.

(2) **Maximum error correction (set $e_c = e_d$):**

$$2e_c \leq d_{\min} - 1 \implies e_c = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor \quad (9)$$

This gives the best possible pure correction capability: $(e_c, e_d) = \left(\left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor, \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor \right)$.

(3) **Fundamental tradeoff:** You cannot simultaneously maximize both detection and correction.

7.4 Quick Reference

Minimum Distance Quick Reference

$$\text{To detect } d \text{ errors requires } d_{\min} \geq d + 1 \quad (10)$$

$$\text{To correct } d \text{ errors requires } d_{\min} \geq 2d + 1 \quad (11)$$

$d_{\min}$	Max Detection ($e_c = 0$)	Max Correction ($e_c = e_d$)
1	0 errors	0 errors
2	1 error	0 errors
3	2 errors	1 error
4	3 errors	1 error
5	4 errors	2 errors
6	5 errors	2 errors
7	6 errors	3 errors

Table 1: Error control capabilities as a function of $d_{\min}$

8 Design Examples

8.1 Design Example 1: Two Codewords from 3-bit Words

Problem: Design a code from 3-bit codewords with 2 codewords. What is the rate of the code? Which codewords should you send? What options exist for error detecting or correcting?

Solution

Rate: We have $k = 1$ data bit encoded into $n = 3$ codeword bits, so:

$$R_c = \frac{k}{n} = \frac{1}{3}$$

Codeword selection: Pick any two codewords that are maximally separated. The pair $\{000, 111\}$ gives maximum separation: $d_{\min} = 3$.

Error control options: Since $d_{\min} = 3$, we need $e_c + e_d \leq 2$ with $e_c \leq e_d$.

e_c	e_d	Decoding Rule
1	1	Correct 1-bit errors: $\{000, 100, 010, 001\} \rightarrow 0$; $\{111, 110, 101, 011\} \rightarrow 1$
0	2	Detect up to 2-bit errors: accept only 000 and 111; retransmit all other received words

8.2 Design Example 2: Four Codewords from 3-bit Words

Problem: Design a code using 3-bit codewords with 4 codewords. What is the rate? Which codewords should you send? What options exist?

Solution

Rate: We have $k = 2$ data bits encoded into $n = 3$ codeword bits, so:

$$R_c = \frac{k}{n} = \frac{2}{3}$$

Codeword selection: Choose 4 codewords that maximize $d_{\min}$. The set $\{000, 011, 101, 110\}$ achieves $d_{\min} = 2$ (every pair differs in exactly 2 positions).

Error control options: Since $d_{\min} = 2$, we need $e_c + e_d \leq 1$ with $e_c \leq e_d$.

Only one sensible option: $(e_c, e_d) = (0, 1)$ — pure error detection.

Codeword	Data
000	00
011	01
101	10
110	11
Others	Retransmit

Note: This is essentially a parity check code. With $d_{\min} = 2$, we can detect 1 error but cannot correct any errors.

8.3 Comparing the Two Designs

Key insight: Packing more codewords into the same n -bit space (higher rate) reduces $d_{\min}$ and weakens error control. There is a fundamental tradeoff between rate and error protection.

Property	Design 1	Design 2
Codeword length n	3	3
Number of codewords	2	4
Data bits k	1	2
Rate R_c	1/3	2/3
$d_{\min}$	3	2
Max detection ($e_c = 0$)	2 errors	1 error
Max correction ($e_c = e_d$)	1 error	0 errors

Table 2: Comparison of the two design examples

9 When to Use Error Correction vs. Detection

9.1 Error Detection is Usually More Efficient

For typical networking scenarios (bidirectional links, low BER), **error detection with retransmission (ARQ)** is preferred because:

- Detection requires fewer redundant bits than correction
- For the same $d_{\min}$, detection handles up to $d_{\min} - 1$ errors while correction handles only $\lfloor (d_{\min} - 1)/2 \rfloor$
- ARQ is 3–5 times more bandwidth-efficient than FEC for typical error rates
- Most network links are bidirectional, so a feedback channel is available

9.2 When to Use Error Correction (FEC)

Error correction is preferred when retransmission is impossible or impractical:

Scenario	Reason
Storage channels (CDs, HDDs, tape)	Errors are permanent; no retransmission possible
Real-time applications (voice, video)	No time for retransmission
Simplex channels (broadcast TV, radio)	No return path for retransmission requests
High-delay channels (satellite, deep space)	Round-trip time makes retransmission too slow

Table 3: Scenarios where FEC is preferred over ARQ

In practice: Many modern systems use **hybrid ARQ (HARQ)**, which combines FEC with retransmission. The receiver first attempts to correct errors using FEC; if correction fails, it requests retransmission.

10 Summary & Key Takeaways

10.1 What We Learned

1. Block Codes

- Add $n - k$ redundant bits to k data bits, creating n -bit codewords

- Designated as (n, k) codes with 2^k valid codewords
- Encoding: $\text{Word} = g(\text{Data})$, often $\text{Word} = \text{Data} \parallel f(\text{Data})$

2. Code Rate

- $R_c = k/n$ measures the fraction of transmitted bits that carry data
- Lower rate means more protection but less throughput

3. Error Control Strategies

- Error correction (FEC): map to nearest codeword
- Error detection (ARQ): reject invalid words, request retransmission
- Combined: correct small errors, detect larger ones

4. Hamming Distance and $d_{\min}$

- Hamming distance: number of differing bit positions
- $d_{\min}$: smallest distance between any pair of codewords
- $d_{\min}$ completely determines error control capability

5. The $d_{\min}$ Theorem

- $e_c + e_d \leq d_{\min} - 1$ with $e_c \leq e_d$
- To detect d errors: need $d_{\min} \geq d + 1$
- To correct d errors: need $d_{\min} \geq 2d + 1$

6. Detection vs. Correction

- Detection is usually more efficient for networking
- Correction is essential when retransmission is impossible

10.2 Looking Ahead: CRC Codes

Next lecture we will study:

- Quantitative comparison of detection vs. correction vs. repetition coding
- **CRC (Cyclic Redundancy Check)** codes — the workhorses of networking error detection
- Polynomial arithmetic over $\text{GF}(2)$
- CRC encoding and decoding algorithms
- Standard CRC polynomials used in Ethernet, WiFi, and other protocols

References

- [1] Kurose, J.F. and Ross, K.W. *Computer Networking: A Top-Down Approach*, 8th Edition. Pearson, 2021.
- [2] Forouzan, B.A. *Data Communications and Networking*, 5th Edition. McGraw-Hill, 2013.
- [3] Wikipedia contributors. “Hamming distance.” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Hamming_distance
- [4] Wikipedia contributors. “Block code.” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Block_code
- [5] Wikipedia contributors. “Error detection and correction.” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Error_detection_and_correction
- [6] Wikipedia contributors. “Parity bit.” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Parity_bit
- [7] Wikipedia contributors. “Repetition code.” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Repetition_code
- [8] Wikipedia contributors. “Forward error correction.” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Forward_error_correction
- [9] Wikipedia contributors. “Automatic repeat request.” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Automatic_repeat_request

Additional Resources

Textbooks:

- Peterson, W.W. and Weldon, E.J. *Error-Correcting Codes*, 2nd Edition. MIT Press, 1972.
- Lin, S. and Costello, D.J. *Error Control Coding*, 2nd Edition. Pearson, 2004.
- Blahut, R.E. *Algebraic Codes for Data Transmission*. Cambridge University Press, 2003.

Standards using error control:

- IEEE 802.3 (Ethernet) — CRC-32 for frame check sequence
- IEEE 802.11 (WiFi) — CRC-32 for FCS, LDPC/BCC for FEC
- 3GPP LTE/5G NR — Turbo codes, LDPC codes, Polar codes
- USB — CRC-5 and CRC-16

Interactive exploration:

- Hamming distance calculator and code design: `lecture_05_06_exploration.py`